

Artificial Intelligence

Inference in First-Order Logic

Christopher Simpkins

Kennesaw State University

1 Propositional vs. First-Order Logic

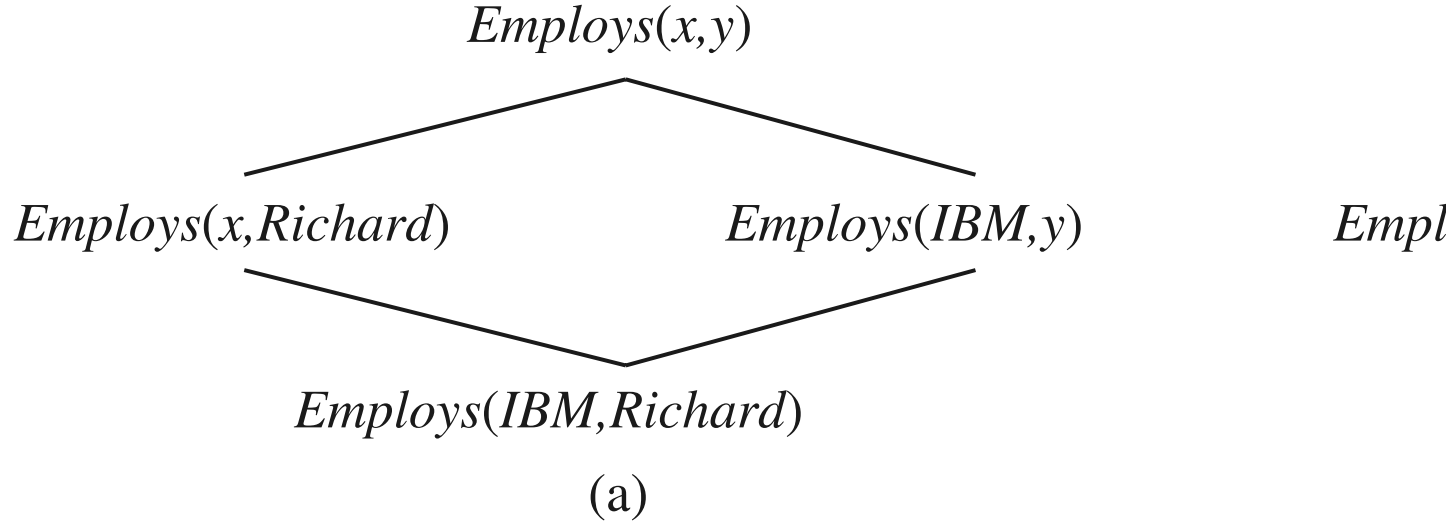
Foo

2 Unification and First-Order Inference

function UNIFY($x, y, \theta = \text{empty}$) **returns** a substitution to make x and y identical, or *failure*
 if $\theta = \text{failure}$ **then return failure**
 else if $x = y$ **then return** θ
 else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)
 else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)
 else if COMPOUND?(x) **and** COMPOUND?(y) **then**
 return UNIFY(ARGS(x), ARGS(y), UNIFY(OP(x), OP(y), θ))
 else if LIST?(x) **and** LIST?(y) **then**
 return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y), θ))
 else return failure

function UNIFY-VAR(var, x, θ) **returns** a substitution
 if $\{var/val\} \in \theta$ for some val **then return** UNIFY(val, x, θ)
 else if $\{x/val\} \in \theta$ for some val **then return** UNIFY(var, val, θ)
 else if OCCUR-CHECK?(var, x) **then return failure**
 else return add $\{var/x\}$ to θ

3 Unification and First-Order Inference

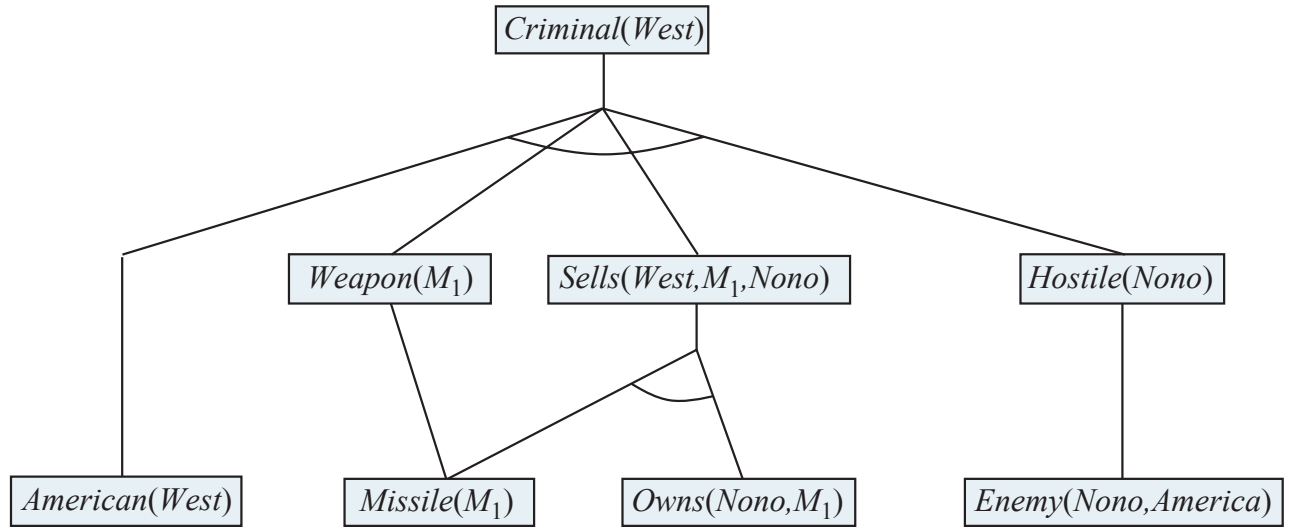


4 Forward Chaining

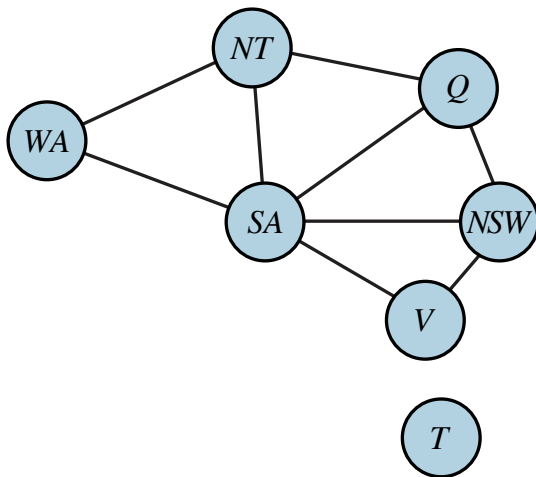
function FOL-FC-ASK(KB, α) **returns** a substitution or *false*
inputs: KB , the knowledge base, a set of first-order definite clauses
 α , the query, an atomic sentence

while *true* **do**
 $new \leftarrow \{\}$ // The set of new sentences inferred on each iteration
 for each *rule* **in** KB **do**
 $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$
 for each θ such that $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$
 for some $p'_1, \dots, p'_n$ in KB
 $q' \leftarrow \text{SUBST}(\theta, q)$
 if q' does not unify with some sentence already in KB or new **then**
 add q' to new
 $\phi \leftarrow \text{UNIFY}(q', \alpha)$
 if ϕ is not *failure* **then return** ϕ
 if $new = \{\}$ **then return** *false*
 add new to KB

5 Forward Chaining



6 Forward Chaining



(a)

$$\begin{aligned}
 &Diff(wa, nt) \wedge Diff(wa, sa) \wedge \\
 &Diff(nt, q) \wedge Diff(nt, sa) \wedge \\
 &Diff(q, nsw) \wedge Diff(q, sa) \wedge \\
 &Diff(nsw, v) \wedge Diff(nsw, sa) \wedge \\
 &Diff(v, sa) \Rightarrow Colorable()
 \end{aligned}$$

$$\begin{aligned}
 &Diff(Red, Blue) \quad Diff(Red, Green) \\
 &Diff(Green, Red) \quad Diff(Green, Blue) \\
 &Diff(Blue, Red) \quad Diff(Blue, Green)
 \end{aligned}$$

(b)

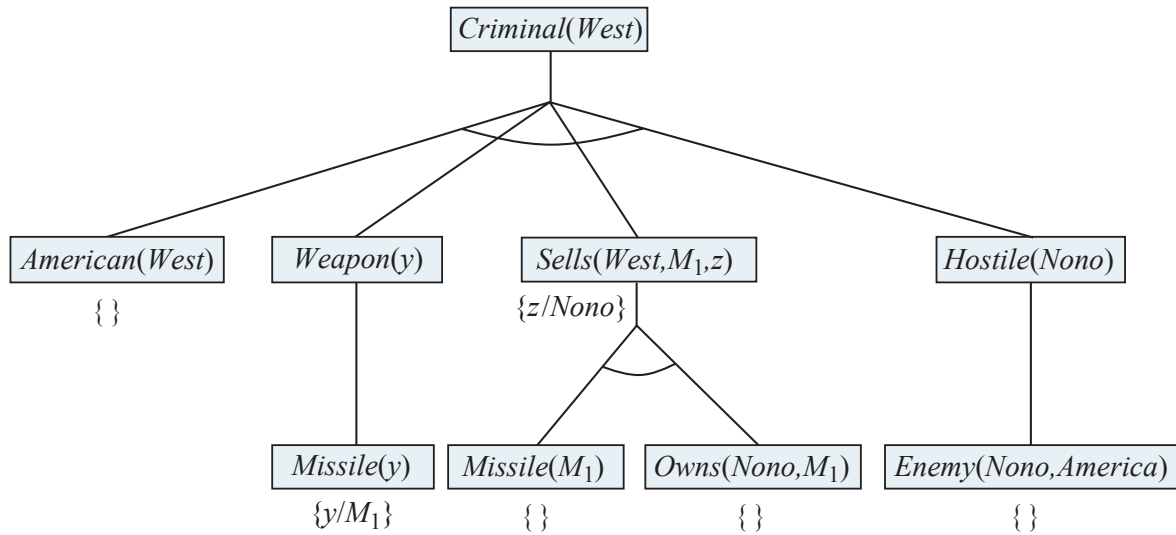
7 Backward Chaining

function FOL-BC-ASK($KB, query$) **returns** a generator of substitutions
return FOL-BC-OR($KB, query, \{\}$)

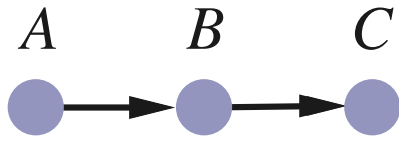
function FOL-BC-OR($KB, goal, \theta$) **returns** a substitution
for each $rule$ **in** FETCH-RULES-FOR-GOAL($KB, goal$) **do**
 $(lhs \Rightarrow rhs) \leftarrow$ STANDARDIZE-VARIABLES($rule$)
for each θ' **in** FOL-BC-AND($KB, lhs, UNIFY(rhs, goal, \theta)$) **do**
yield θ'

function FOL-BC-AND($KB, goals, \theta$) **returns** a substitution
if $\theta = failure$ **then return**
else if LENGTH($goals$) = 0 **then yield** θ
else
 $first, rest \leftarrow$ FIRST($goals$), REST($goals$)
for each θ' **in** FOL-BC-OR($KB, SUBST(\theta, first), \theta$) **do**
for each θ'' **in** FOL-BC-AND($KB, rest, \theta'$) **do**
yield θ''

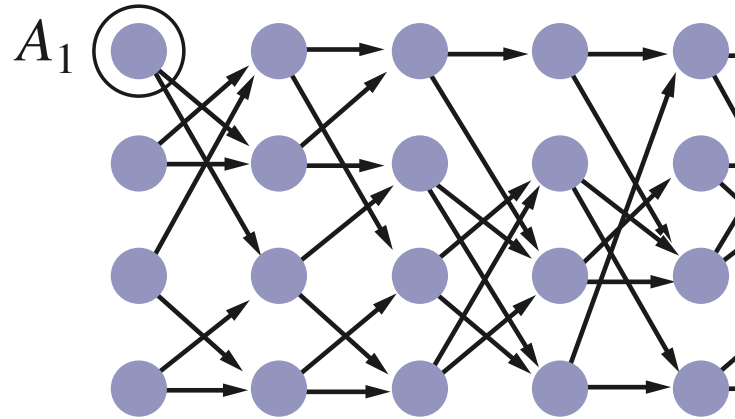
8 Backward Chaining



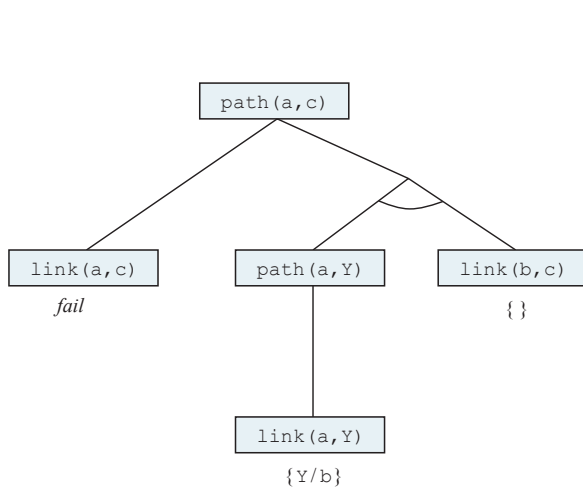
9 Logic Programming



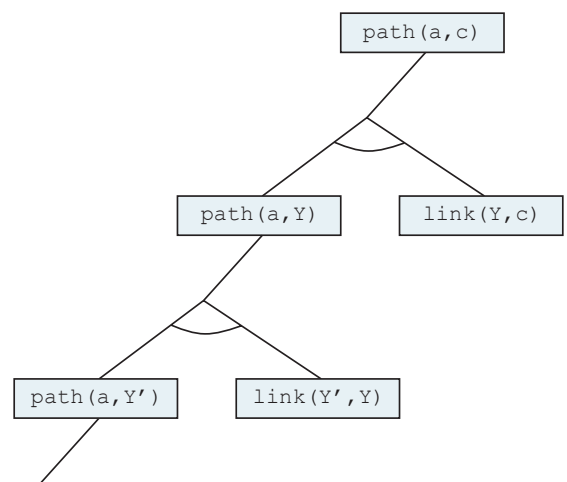
(a)



10 Logic Programming

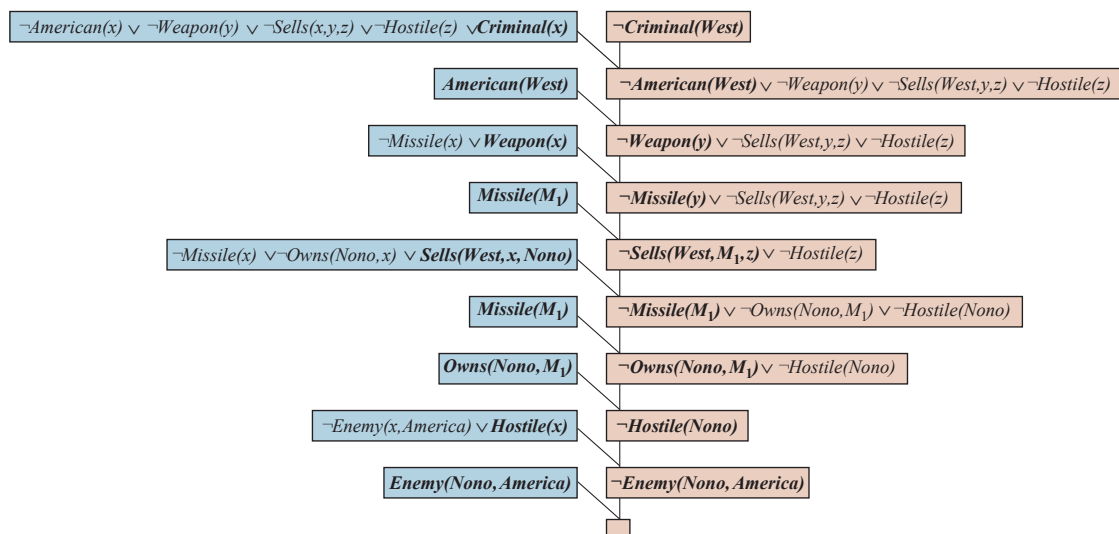


(a)

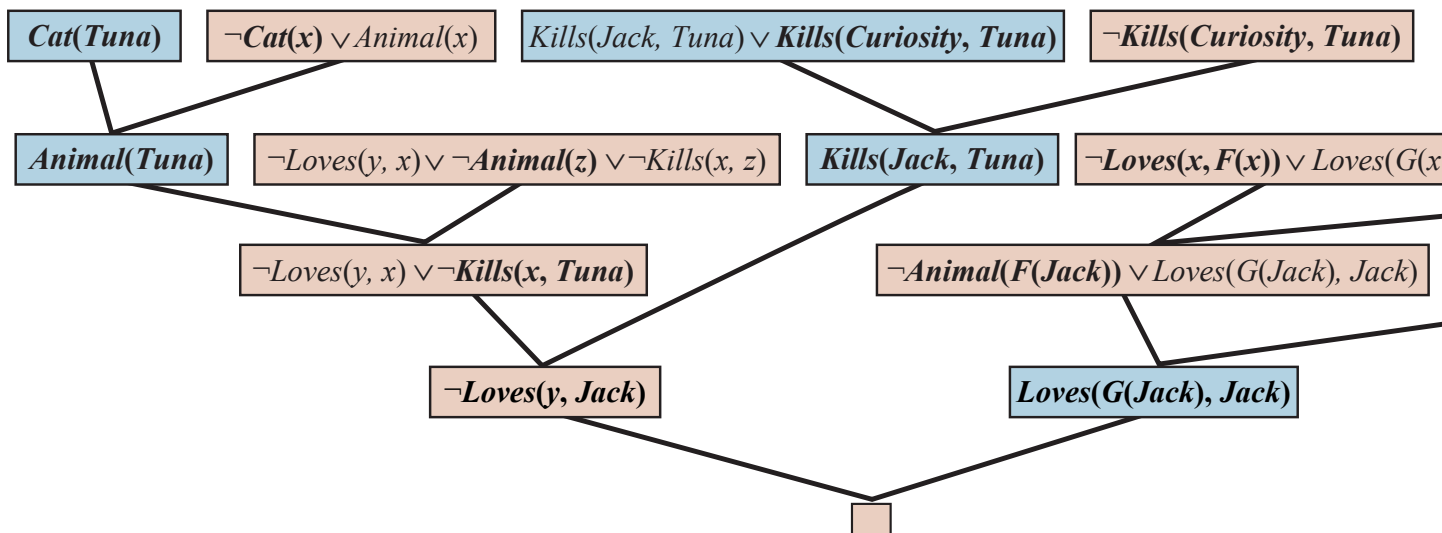


(b)

11 Resolution



12 Resolution



13 Completeness

Any set of sentences S is representable in clausal form

Assume S is unsatisfiable, and in clausal form

Some set S' of ground instances is unsatisfiable

Resolution can find a contradiction in S'

There is a resolution proof for the contradiction in S'

Herbrand's theorem

Ground resolution
theorem

Lifting lemma

14 Gödel's Incompleteness Theorem

Foo